

บทที่ 12

คลาส

สำหรับบทสุดท้ายของตำราเล่มนี้ผู้เขียนต้องการแนะนำให้ผู้เรียนได้เรียนรู้เกี่ยวกับกระบวนการในการเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming) ด้วยภาษาไพธอน ซึ่งการเขียนโปรแกรมเชิงวัตถุนั้นเป็นวิธีการหนึ่งที่ช่วยให้นักพัฒนาโปรแกรมสามารถเขียนโปรแกรมได้อย่างรวดเร็วและมีประสิทธิภาพเป็นอย่างมากในยุคปัจจุบันด้วยข้อดีหลายประการ เช่น ช่วยให้ผู้พัฒนาโปรแกรมสามารถเปลี่ยนแปลงโปรแกรมที่เขียนมาแล้วได้ง่ายขึ้น และโปรแกรมที่ถูกเขียนขึ้นมาแล้วสามารถนำกลับไปใช้ในโปรแกรมที่เขียนขึ้นใหม่ได้ โดยอาจมีการแก้ไขเพียงเล็กน้อยหรือไม่แก้ไขเลย (Reusable) เป็นต้น วิธีการเขียนโปรแกรมเชิงวัตถุผู้พัฒนาโปรแกรมสามารถสร้าง “วัตถุ” (Object) จาก “คลาส” (Classes) ที่จำลองถึงสิ่งต่าง ๆ หรือสถานการณ์ต่าง ๆ ที่มีอยู่ในโลกแห่งความเป็นจริงให้อยู่ในรูปแบบโปรแกรมคอมพิวเตอร์ เพื่อให้คอมพิวเตอร์สามารถนำไปประมวลผลเพื่อให้ได้คำตอบสำหรับปัญหาที่เราต้องการแก้ไขได้ต่อไปได้

ดังนั้นในบทเรียนนี้ผู้เขียนจึงได้เน้นเจาะจงไปที่ “คลาส” ซึ่งถือว่าเป็นต้นแบบของวัตถุ การสร้างและใช้งานคลาส การทำงานกับคลาสและอินสแตนซ์ ตลอดจนการสืบทอดคุณสมบัติจากคลาส เพื่อเป็นการแนะนำให้ผู้เรียนได้เริ่มเข้าใจแนวคิด จุดเด่น ตลอดจนสามารถมองเห็นถึงความแตกต่างระหว่างกระบวนการพัฒนาโปรแกรมแบบเชิงวัตถุ และการเขียนโปรแกรมแบบกระบวนการคำสั่ง (Procedural Programming) ดังที่ผู้เรียนได้เรียนรู้จากบทเรียนก่อนหน้านี้ได้อย่างชัดเจนและสามารถนำไปประยุกต์ใช้ได้ต่อไป

12.1. การสร้างและใช้งานคลาส

จากแนวคิดของการเขียนโปรแกรมเชิงวัตถุที่ผู้เขียนได้แนะนำให้ผู้เรียนได้รับทราบแล้วว่า เราสามารถที่จะสร้างแบบจำลองของทุก ๆ สิ่ง รวมทั้งเหตุการณ์ต่าง ๆ ด้วยการสร้างคลาสได้ทั้งสิ้น โดยในหัวข้อนี้ผู้เขียนจะเริ่มต้นด้วยการแนะนำให้ผู้เรียนได้ทดลองสร้างคลาสแบบง่ายด้วยการจำลองแบบสิ่งมีชีวิตที่อยู่ใกล้ ๆ ตัวของเรานั้นคือ สุนัข (Dog) ซึ่งเป็นสุนัขที่มีลักษณะโดยทั่ว ๆ ไป และก่อนที่จะเริ่มจำลองแบบของสุนัขด้วยโปรแกรมคอมพิวเตอร์ ผู้เรียนลองคิดถึงรายละเอียดของสุนัขที่เรารู้จักโดยทั่วไปว่าจะต้องมีรายละเอียดอะไรบ้าง เช่น สุนัขจะต้องมี ชื่อ และอายุ และจะต้องมีพฤติกรรมอย่างไร เช่น สุนัขจะต้องสามารถ นั่ง และกลิ้งไปกลิ้งมาได้ เป็นต้น

เมื่อเราได้ข้อสรุปแล้วว่าสุนัขที่เรารู้จักจะต้องมี ชื่อ, อายุ และพฤติกรรมทั่วไปคือ มีความสามารถ นั่ง, กลิ้งไปกลิ้งมาแล้วนั้น เราจะนำสิ่งเหล่านี้ไปสร้างเป็นคลาสชื่อว่า “Dog”

ซึ่งคลาสนี้จะเป็นสิ่งที่บอกให้โปรแกรมภาษาไพธอนได้รับทราบว่าเราจะแทนสุนัขโดยทั่วไปด้วยโปรแกรมคอมพิวเตอร์อย่างไร และเมื่อคลาสถูกสร้างขึ้นแล้ว เราจะนำคลาสนี้ไปสร้างเป็นวัตถุหรือ อินสแตนซ์ (Instances) ซึ่งอินสแตนซ์นี้จะบ่งชี้ถึงสุนัขเพียงตัวใดตัวหนึ่งอย่างเจาะจงเท่านั้น

12.1.1. การสร้างคลาสในภาษาไพธอน

เพื่อให้ผู้เรียนได้เข้าใจถึงการสร้างคลาส และการจำลองแบบสิ่งมีชีวิตด้วยคลาสอย่างชัดเจน เรลองมาทำความเข้าใจคลาส “Dog” ผ่านโปรแกรมภาษาไพธอนดังต่อไปนี้

```
1. class Dog():
2.     """การจำลองสุนัขทั่วไปด้วยคลาส"""
3.     def __init__(self, name, age):
4.         self.name = name
5.         self.age = age
6.
7.     def sit(self):
8.         """การจำลองพฤติกรรมการนั่ง"""
9.         print("{} is now sitting.".format(self.name.title()))
10.
11.    def roll_over(self):
12.        """การจำลองพฤติกรรมการวิ่ง"""
13.        print("{} is rolled over!".format(self.name.title()))
```

จากตัวอย่าง คลาส Dog ด้านบนสามารถอธิบายได้ดังนี้

บรรทัดที่ 1 เป็นการประกาศคลาสชื่อว่า Dog

บรรทัดที่ 3 ฟังก์ชันที่อยู่ภายในคลาสเราจะเรียกว่า “เมธอด” (Method) โดยเมธอด `__init__()` นี้เป็นเมธอดพิเศษที่จะถูกโปรแกรมภาษาไพธอนเรียกใช้งานโดยอัตโนมัติทุกครั้ง ที่นำเอาคลาสนี้ไปสร้างอินสแตนซ์ โดยมีข้อสังเกตว่าเมธอดของคลาสนี้จะเริ่มต้นและถูกปิดท้ายด้วยเครื่องหมาย “_” (Underscores) สองตัว โดยหลักการตั้งชื่อเมธอดแบบนี้มีประโยชน์ในการช่วยป้องกันไม่ให้ชื่อเมธอดที่ถูกสงวนเอาไว้ ถูกนำไปใช้ซ้ำซ้อนกันกับชื่อที่ทางผู้พัฒนาเป็นคนตั้งขึ้น ในภายหลังนั่นเอง เมธอด `__init__()` ของคลาส Dog ที่เราสร้างขึ้นนั้นจะรองรับพารามิเตอร์ที่จะต้องถูกส่งเข้ามาทั้งหมด 3 ตัวคือ `self`, `name` และ `age` แต่มีข้อสังเกตและจำเป็นที่ผู้เรียนต้องทราบสำหรับพารามิเตอร์ `self` คือ เมื่อเมธอด `__init__()` ถูกเรียกใช้งานตอนที่เรานำคลาส Dog ไปสร้างอินสแตนซ์ การเรียกใช้งานเมธอดนั้นจะมีการส่งค่าให้กับพารามิเตอร์ `self` โดยอัตโนมัติ รวมทั้งทุก ๆ การเรียกใช้งานทุก ๆ เมธอดของคลาส Dog ก็จะมีการส่งผ่านค่าให้กับพารามิเตอร์ `self` โดยอัตโนมัติด้วยเช่นกัน

บรรทัดที่ 4 ตัวแปรภายในคลาสถูกประกาศขึ้นด้วยการใช้ `self` นำหน้า ซึ่งแสดงถึงว่าตัวแปรใด ๆ ที่นำหน้าด้วย `self` จะมีขอบเขตการเรียกใช้งานได้ในทุก ๆ เมธอดภายในคลาสนอกจากนั้นยังสามารถเข้าถึงผ่านอินสแตนซ์ใด ๆ ก็ตามที่ถูกสร้างจากคลาสนี้อีกด้วย ตัวอย่างเช่น

`self.name=name` คือการรับค่าที่เก็บไว้ในพารามิเตอร์ `name` และทำการเก็บค่าเอาไว้ในตัวแปรของคลาสคือ `self.name` ซึ่งกระบวนการนี้จะเกิดขึ้นตอนที่อินสแตนซ์ถูกสร้างขึ้น และค่าข้อมูลที่ถูกเก็บไว้ในตัวแปร `self.name` จะสามารถถูกเข้าถึงผ่านอินสแตนซ์นั้นได้อีกด้วย โดยเราจะเรียกตัวแปรใด ๆ ของคลาสที่สามารถเข้าถึงผ่านอินสแตนซ์ว่า “แอตทริบิวต์” (Attributes)

บรรทัดที่ 7 เป็นการประกาศเมธอดของคลาส `Dog` โดยในคลาส `Dog` นี้มี 2 เมธอดคือ `sit()` และ `roll_over()` เนื่องจากเมธอดทั้งสองนี้ไม่ต้องการใช้ข้อมูลใด ๆ เพิ่มเติมสำหรับการทำงานจึงไม่มีการประกาศรับพารามิเตอร์จากภายนอก แต่ในการประกาศเมธอดของคลาสนั้นยังจำเป็นต้องการกำหนดพารามิเตอร์หนึ่งตัวคือ `self` เอาไว้

ดังนั้นเพื่อความง่ายต่อการจดจำ และความเข้าใจของผู้เรียน ผู้เขียนจึงขอสรุปว่าทุก ๆ เมธอดของคลาสนั้นจะต้องมีการประกาศพารามิเตอร์ `self` เอาไว้เป็นพารามิเตอร์ตัวแรกสุดเสมอ ถึงแม้ว่าเมธอดนั้นจะไม่มีค่าจำเป็นต้องรับพารามิเตอร์ใด ๆ จากภายนอกเลยก็ตาม

12.1.2. การสร้างอินสแตนซ์จากคลาส

จากหัวข้อที่ 12.1.1 ผู้เรียนได้เรียนรู้การสร้างคลาสด้วยภาษาไพธอน และรับทราบรายละเอียดเกี่ยวกับคลาสเบื้องต้นไปแล้วนั้น ซึ่งจะเห็นได้ว่า “คลาส” เปรียบเสมือนชุดของคำสั่งโปรแกรมที่แสดงรายละเอียดของสิ่งที่เราต้องการจำลองแบบว่าสิ่งนั้นมีรายละเอียด และพฤติกรรมที่สามารถแสดงออกได้อย่างไรบ้าง โดยการนำคลาสที่เราสร้างขึ้นมาใช้งานนั้น ทำได้โดยการสร้างอินสแตนซ์ขึ้นมาจากคลาสนั้นนั่นเอง

สำหรับคลาส `Dog` เปรียบเสมือนการจำลองแบบของสุนัขทั่ว ๆ ไปที่ไม่ได้บ่งชี้อย่างเจาะจงว่าเป็นสุนัขตัวใดตัวหนึ่ง แต่การนำเอาคลาส `Dog` มาสร้างอินสแตนซ์จะหมายถึงการสร้างวัตถุที่เจาะจงถึงสุนัขเพียงแค่ตัวเดียวเท่านั้น โดยคลาสหนึ่งคลาสสามารถนำไปสร้างอินสแตนซ์ได้อย่างไม่จำกัด และทำได้โดยการดูจากชุดของคำสั่งด้านล่างนี้

```
1. my_dog = Dog("Kanoon", 3)
2.
3. print("My dog's name is {}".format(my_dog.name.title()))
4.
5. print("My dog is {} year old.".format(my_dog.age))
```

[output]

```
My dog's name is Kanoon.
My dog is 3 year old.
```

การสร้างอินสแตนซ์จากคลาสในภาษาไพธอนนั้นสามารถทำได้ดังแสดงในบรรทัดที่ 1 ของตัวอย่างด้านบน ซึ่งเราทำการสร้างและเก็บอินสแตนซ์ของคลาส `Dog` ไว้ในตัวแปร `my_dog` ซึ่งเป็นการบอกให้ภาษาไพธอนทราบว่าเราต้องการสร้างสุนัขชื่อว่า “Kanoon” (ขนุน) ซึ่งมีอายุ 3 ปี

โดยเมื่อโปรแกรมภาษาไพธอนทำการประมวลผลชุดคำสั่งในบรรทัดที่ 1 จะทำการเรียกเมธอด `__init__()` ของคลาส `Dog` ให้ทำงาน ซึ่งจะเป็นการกำหนดค่าให้กับแอตทริบิวต์ของคลาส คือ `self.name` และ `self.age` นั้นเอง

12.1.3. การเข้าถึงแอตทริบิวต์ของอินสแตนซ์

การเข้าถึงแอตทริบิวต์ของอินสแตนซ์สามารถทำได้ดังตัวอย่างโค้ดโปรแกรมด้านบนในบรรทัดที่ 2 ซึ่งเป็นการเข้าถึงข้อมูลที่เก็บไว้ในแอตทริบิวต์ `name` ของอินสแตนซ์ `my_dog` ซึ่งสามารถเขียนโปรแกรมในรูปแบบ ดังนี้

```
1. >>> my_dog.name
```

สำหรับบรรทัดที่ 5 นั้นเราสามารถใช่วิธีเดียวกันนี้ในการเข้าถึงแอตทริบิวต์ `age` ของ `my_dog` ได้เช่นกัน

12.1.4. การเรียกเมธอด

หลังจากที่ทำการสร้างอินสแตนซ์จากคลาส `Dog` แล้วเราสามารถใช้อัตลักษณ์จุด (“.”) เพื่อเรียกใช้เมธอดใด ๆ ที่อยู่ในคลาส `Dog` เพื่อทำให้อินสแตนซ์สุนัขของเราแสดงพฤติกรรมนั่งและนอนเกลือกกลิ้งได้ ดังตัวอย่างด้านล่าง

```
1. my_dog.sit()
2.
3. my_dog.roll_over()
```

[output]

```
Kanoon is now sitting.
Kanoon is rolled over!
```

ในการเรียกเมธอดสามารถทำได้โดยผ่านการอ้างถึงชื่อของอินสแตนซ์ คือ `my_dog` และเมธอดที่เราต้องการเรียกโดยคั่นด้วยจุดระหว่างชื่อและเมธอด เมื่อโปรแกรมภาษาไพธอนทำการอ่านคำสั่ง `my_dog.sit()` จะมองหาเมธอด `sit()` ในคลาส `Dog` และรันโค้ดนั้น ซึ่งก็จะตีความบรรทัด `my_dog.roll_over()` ในลักษณะเดียวกัน

12.1.5. การสร้างหลายอินสแตนซ์จากคลาสเดียว

จากเนื้อหาที่ผ่านมาของบทเรียนนี้ ผู้เรียนได้ทราบแล้วว่าเราสามารถสร้างอินสแตนซ์ได้จากคลาส ซึ่งอินสแตนซ์ที่เกิดขึ้นนั้นจะเป็นการจำเพาะเจาะจงลงไปถึงสิ่งที่เราจำลองแบบ เช่น อินสแตนซ์ `my_dog` จากตัวอย่างนั้นจะหมายถึงเพียงตัวเดียวที่ชื่อ ขนุน ที่มีอายุ 3 ปีเท่านั้น ดังนั้นสิ่งที่ผู้เรียนอาจจะยังคงมีข้อสงสัยอยู่ว่า แล้วคลาสหนึ่งคลาสจะสามารถนำไปสร้างอินสแตนซ์ได้จำนวนเท่าใด ซึ่งคำตอบของข้อสงสัยนี้คือ เราสามารถสร้างอินสแตนซ์จากคลาสได้จำนวนมากเท่าที่เราต้องการ

โดยสามารถแสดงการสร้างหลายอินสแตนซ์จากคลาสเดียวดังตัวอย่างด้านล่างต่อไปนี้

```
1. thungtong = Dog("Thungtong", 8)
2. thunggnern = Dog("Thunggnern", 9)
3.
4. print("My dog's name is {}".format(thungtong.name.title()))
5. print("My dog's name is {}".format(thunggnern.name.title()))
```

[output]

```
My dog's name is Thungtong.
My dog's name is Thunggnern.
```

12.2. การทำงานกับคลาสและอินสแตนซ์

การทำงานกับคลาสและอินสแตนซ์ สิ่งแรกที่คุณเรียนจะต้องมีความรู้ ความเข้าใจและสามารถลงมือปฏิบัติได้ คือ การแก้ไขค่าแอตทริบิวต์ของอินสแตนซ์ที่เรากำลังทำงานอยู่ด้วย ซึ่งสามารถทำได้โดยการแก้ไขค่าแอตทริบิวต์โดยตรง หรือแก้ไขผ่านช่องทางเฉพาะที่จัดเตรียมเอาไว้เท่านั้น

12.2.1. สร้างคลาส Car

เพื่อให้สามารถทำการอธิบายถึงกระบวนการแก้ไขค่าแอตทริบิวต์ของอินสแตนซ์ได้ชัดเจนมากขึ้น ผู้เขียนจึงขอเริ่มต้นด้วยการให้คุณเรียนทำการสร้างคลาส “รถยนต์” (Car) ขึ้นมา โดยรายละเอียดข้อมูลของคลาส Car นั้นประกอบไปด้วย ยี่ห้อรถยนต์, โมเดล และปีที่ผลิต และมีเมธอดที่ใช้สำหรับรายงานรายละเอียดของรถยนต์คันนั้น ดังตัวอย่างด้านล่าง

```
1. class Car():
2.     """คลาสสำหรับจำลองรถยนต์"""
3.     def __init__(self, make, model, year):
4.         """กำหนดค่าเริ่มต้นให้กับแอตทริบิวต์ของคลาสรถยนต์"""
5.         self.make = make
6.         self.model = model
7.         self.year = year
8.
9.     def get_descriptive_name(self):
10.        """ส่งคืนคำรายละเอียดต่าง ๆ ของรถยนต์"""
11.        long_name = str(self.year)+" "+self.make+" "+self.model
12.        return long_name.title()
13.
14. my_new_car = Car('Toyota', 'Altis', 2016)
15.
16. print(my_new_car.get_descriptive_name())
```

[output]

```
2016 Toyota Altis
```

ในคลาส Car เราทำการกำหนดเมธอด `__init__()` ด้วยพารามิเตอร์ `self` ก่อน เช่นเดียวกับที่เราทำกับคลาส Dog ซึ่งเป็นตัวอย่างก่อนหน้านี้ นอกจากนี้ยังมีพารามิเตอร์อีก 3 ตัว ประกอบไปด้วย `make`, `model` และ `year` โดย เมธอด `__init__()` ทำการรับพารามิเตอร์ เหล่านี้และเก็บไว้ในแอตทริบิวต์ที่เชื่อมกับอินสแตนซ์ที่ถูกสร้างจากคลาสนี้ โดยทุกครั้งที่เราทำการ สร้างอินสแตนซ์จากคลาส Car จะต้องการระบุชื่อ รุ่น และปีที่ผลิตทุกครั้งเสมอ

จากคลาส Car ด้านบนเราทำการสร้างเมธอดสำหรับเรียกดูข้อมูลเกี่ยวกับรถยนต์แต่ละคัน คือเมธอด `get_descriptive_name()` ซึ่งจะทำการรวมแอตทริบิวต์ `make`, `model` และ `year` เข้าไว้เป็นสตริงและส่งค่าออกมา ซึ่งเมธอดนี้จะช่วยให้เราไม่ต้องทำการพิมพ์ค่าแอตทริบิวต์ออกมา ทีละรายการซึ่งถือว่าเป็นการอำนวยความสะดวกแก่ผู้ใช้งานโปรแกรมเมื่อต้องการทราบรายละเอียด เกี่ยวกับอินสแตนซ์นั้น

12.2.2. การตั้งค่าเริ่มต้นสำหรับแอตทริบิวต์ของคลาส

เพื่อให้คลาส Car ที่เราสร้างขึ้นมีความน่าสนใจ และสอดคล้องกับสิ่งที่เรากำลังจะได้เรียนรู้ ในเนื้อหาใหม่มากยิ่งขึ้น ผู้เขียนจะทำการเพิ่มแอตทริบิวต์ที่จะต้องเปลี่ยนแปลงข้อมูลอยู่ตลอดเวลา โดยเราจะเพิ่มแอตทริบิวต์ที่ใช้เก็บข้อมูลระยะทางสะสมของรถยนต์ (car's overall mileage) แต่ละ คันเข้าไปในคลาส Car และเพิ่มเมธอด `read_odometer()` เข้าไปในคลาสเพื่อช่วยให้เราเข้าถึง ข้อมูลระยะสะสมของรถยนต์จากอินสแตนซ์ได้ ดังตัวอย่างด้านล่างนี้

```
1. class Car():
2.     """คลาสสำหรับจำลองรถยนต์"""
3.     def __init__(self, make, model, year):
4.         """กำหนดค่าเริ่มต้นให้กับแอตทริบิวต์ของคลาสรถยนต์"""
5.         self.make = make
6.         self.model = model
7.         self.year = year
8.         self.odometer = 0
9.
10.    def get_descriptive_name(self):
11.        """ส่งคืนคำรายละเอียดต่าง ๆ ของรถยนต์"""
12.        long_name = str(self.year)+" "+self.make+" "+self.model
13.        return long_name.title()
14.
15.    def read_odometer(self):
16.        """แสดงรายละเอียดระยะสะสมของรถยนต์"""
17.        print("This car has " + str(self.odometer) + " miles")
18.
19. my_new_car = Car('Toyota', 'Altis', 2016)
20.
21. print(my_new_car.get_descriptive_name())
22. my_new_car.read_odometer()
```

```
[output]
2016 Toyota Altis
This car has 0 miles
```

จากตัวอย่างโค้ดโปรแกรมมีข้อน่าสังเกตดังนี้ เนื่องจากเมธอด `__init__()` ไม่ได้มีการรับค่าพารามิเตอร์เพื่อใช้สำหรับกำหนดค่าเริ่มต้นให้กับแอตทริบิวต์ `self.odometer` ดังนั้น จึงมีความจำเป็นต้องกำหนดค่าเริ่มต้นให้แก่ `self.odometer` เอง ซึ่งจากตัวอย่างเราได้กำหนดค่าเริ่มต้นให้เท่ากับ 0 ดังแสดงในบรรทัดที่ 8

12.2.3. การแก้ไขค่าแอตทริบิวต์โดยตรง

วิธีที่ง่ายที่สุดในการแก้ไขค่าของแอตทริบิวต์ คือการเข้าถึงแอตทริบิวต์โดยตรงผ่านอินสแตนซ์ โดยตัวอย่างด้านล่างนี้ แสดงถึงการเปลี่ยนแปลงค่าระยะทางสะสมของรถยนต์จาก 0 ให้กลายเป็น 23 ด้วยการแก้ไขโดยตรง

```
1. my_new_car.odometer = 23
2.
3. my_new_car.read_odometer()
```

```
[output]
This car has 23 miles
```

จากตัวอย่างเราใช้จุด (".") เพื่อเข้าถึงแอตทริบิวต์ `odometer` ของรถยนต์ และทำการกำหนดค่าให้ใหม่โดยตรง โดยบรรทัดที่ 1 นั้นเป็นการบอกให้โปรแกรมภาษาไพธอนค้นหาแอตทริบิวต์ `odometer` ในอินสแตนซ์ `my_new_car` และทำการกำหนดค่าของแอตทริบิวต์นั้นให้เป็น 23

12.2.4. การแก้ไขค่าของแอตทริบิวต์ผ่านเมธอด

ถึงแม้ว่าการแก้ไขค่าของแอตทริบิวต์โดยตรงนี้จะง่ายและสะดวกต่อการทำงานของผู้พัฒนาโปรแกรม แต่ถ้าหากพิจารณาในเรื่องของความปลอดภัยเกี่ยวกับข้อมูลแล้วนั้น ข้อมูลบางอย่างก็ยังไม่สมควรที่จะปล่อยให้มีการเข้าถึงได้โดยตรง และปล่อยให้ถูกแก้ไขได้จากภายนอกคลาสโดยที่ไม่มีการควบคุมใด ๆ ดังนั้นการจัดการปรับเปลี่ยนหรือตั้งค่าของแอตทริบิวต์ผ่านเมธอด ซึ่งเป็นการปรับเปลี่ยนค่าจากภายใน และควบคุมให้มีการปรับเปลี่ยนเฉพาะบางแอตทริบิวต์ที่สามารถปรับเปลี่ยนค่าได้เท่านั้น ซึ่งในกรณีนี้จะช่วยในการควบคุมข้อผิดพลาด และมีการรักษาความปลอดภัยของข้อมูลภายในคลาสที่เหมาะสมกว่า

โดยตัวอย่างด้านล่างนี้ แสดงให้ผู้เรียนเห็นถึงการแก้ไขเปลี่ยนแปลงแอตทริบิวต์ `odometer` จากภายในคลาส `Car` ผ่านเมธอด `update_odometer()` ด้วยการส่งค่าใหม่ผ่านทางอากิวเมนต์ตอนที่เรียกเมธอด ซึ่งทำได้ดังนี้

```

1. class Car():
2.     ..
3.     ..
4.     def update_odometer(self, mileage):
5.         """กำหนดค่าระยะทางสะสมใหม่จากพารามิเตอร์"""
6.         self.odometer = mileage
7.
8. my_new_car = Car('Toyota', 'Altis', 2016)
9. my_new_car.read_odometer()
10.
11. my_new_car.update_odometer(50)
12. my_new_car.read_odometer()

```

[output]

```

2016 Toyota Altis
This car has 0 miles
This car has 50 miles

```

และเพื่อให้ผู้เรียนได้เข้าใจอย่างลึกซึ้งเพิ่มอีกว่า การเปลี่ยนค่าแอตทริบิวต์จากภายในคลาสผ่านกลไกของเมธอดนั้นแตกต่างและมีประโยชน์ต่อการควบคุมความถูกต้องต่อการทำงานของโปรแกรมมากกว่าการปล่อยให้มีการแก้ไขค่าแอตทริบิวต์โดยตรงจากภายนอกอย่างไร ลองให้ผู้เรียนคิดถึงความเป็นจริงว่าระยะทางสะสมของรถยนต์ทุกคันนั้น ควรเป็นค่าที่เพิ่มขึ้นเพียงอย่างเดียวเท่านั้น ไม่สามารถลดค่าลงได้ ดังนั้นค่าที่เป็นค่าใหม่ของแอตทริบิวต์ odometer จะต้องเป็นค่าที่มากขึ้นกว่าเดิมทุกครั้งที่มีการเปลี่ยนแปลงค่าใหม่ให้กับแอตทริบิวต์ odometer ซึ่งตัวอย่างข้างล่างนี้จะแสดงให้เห็นถึงการใช้กลไกภายในเมธอดควบคุม และตรวจสอบความถูกต้องของการกำหนดค่าให้กับแอตทริบิวต์ภายในคลาส

```

1. class Car():
2.     ..
3.     ..
4.     def update_odometer(self, mileage):
5.         """กำหนดค่าระยะทางสะสมใหม่จากพารามิเตอร์"""
6.         if mileage > self.odometer:
7.             self.odometer = mileage
8.         else:
9.             print("คุณไม่สามารถลดค่าระยะสะสมได้")
10.
11. my_new_car.update_odometer(50)
12. my_new_car.read_odometer()
13.
14. my_new_car.update_odometer(30)
15. my_new_car.read_odometer()

```

[output]

```

This car has 0 miles
This car has 50 miles
คุณไม่สามารถลดค่าระยะสะสมได้
This car has 50 miles

```

จากตัวอย่างนี้ ทุกครั้งที่มีการกำหนดค่าใหม่ให้กับแอตทริบิวต์ `odometer` ของคลาส `Car` ผ่านเมธอด `update_odometer()` โปรแกรมจะทำการตรวจสอบทุกครั้งว่าค่าใหม่ที่ถูกส่งเข้ามานั้น มีค่ามากกว่าค่าเดิมที่ถูกกำหนดไว้ก่อนหน้านี้หรือไม่ ด้วยกระบวนการนี้จะไม่สามารถลดค่าระยะสะสมของรถยนต์แต่ละคันได้เลย ซึ่งจะส่งผลให้โปรแกรมสามารถทำงานได้อย่างถูกต้อง และสอดคล้องกับสิ่งที่เราต้องการจริง ๆ

12.3. การสืบทอด

เมื่อผู้เรียนได้รู้จักกับคลาส ตลอดจนสามารถสร้างและใช้งานคลาสเป็นที่เรียบร้อยแล้ว จากเนื้อหาที่ผ่านมาของบทเรียนนี้ หัวข้อนี้ผู้เขียนจะกล่าวถึงคุณสมบัติที่สำคัญอีกประการหนึ่งของการเขียนโปรแกรมเชิงวัตถุที่ทำให้การพัฒนาโปรแกรมในปัจจุบันเป็นไปอย่างรวดเร็ว และมีประสิทธิภาพนั่นคือ “การสืบทอด” ซึ่งผู้เขียนจะกล่าวถึง ความหมายของการสืบทอด การสืบทอดในภาษาไพธอน การกำหนดแอตทริบิวต์และเมธอดให้คลาสลูก การสืบทอดจากหลายคลาส การลบล้างหรือแก้ไขเมธอดที่ได้รับสืบทอดจากคลาสแม่ ซึ่งเนื้อหาและรายละเอียดเหล่านี้เมื่อผู้เรียนได้เรียนรู้แล้ว จะทำให้ผู้เรียนจะสามารถนำเอาแนวคิดและกระบวนการในการเขียนโปรแกรมเชิงวัตถุไปประยุกต์ใช้ในการพัฒนาโปรแกรมเพื่อแก้ปัญหาในชีวิตจริงได้อย่างมีประสิทธิภาพมากยิ่งขึ้นต่อไป

12.3.1. ความหมายของการสืบทอด

คุณสมบัติที่สำคัญอีกอย่างหนึ่งของการเขียนโปรแกรมเชิงวัตถุซึ่งทำให้การพัฒนาโปรแกรมในยุคปัจจุบันมีประสิทธิภาพและสามารถพัฒนาได้อย่างรวดเร็ว คือ การสืบทอด (Inheritance) โดยผู้พัฒนาสามารถสร้างคลาสใหม่จากคลาสเดิมที่มีอยู่แล้ว ซึ่งไม่ใช่การนำเอาคลาสเดิมมาแก้ไขโค้ดโปรแกรมใหม่ แต่เป็นการนำคลาสเดิมมาสืบทอดคุณสมบัติให้คลาสใหม่ที่สร้างขึ้นโดยไม่ต้องไปแก้ไขเปลี่ยนแปลงคลาสเดิมแต่อย่างใด โดยคลาสใหม่ที่ได้นี้ ต่อไปเราจะเรียกว่า “คลาสลูก” นี้จะมีแอตทริบิวต์ และพฤติกรรมเหมือนกับคลาสต้นแบบที่สืบทอดมา หรือต่อไปเราจะเรียกว่า “คลาสแม่” ทุกประการ นอกจากนี้คลาสลูกยังมีอิสระในการกำหนดแอตทริบิวต์และเมธอดใหม่ของตัวเองได้ด้วย

12.3.2. การสืบทอดในภาษาไพธอน

การสืบทอดในภาษาไพธอนสามารถทำได้สามารถทำได้โดยใช้ไวยากรณ์ดังนี้

```
1. class BaseClass:
2.     Body of base class
3.
4. class DerivedClass(BaseClass):
5.     Body of derived class
```

ไวยากรณ์ของภาษาไพธอนสำหรับการสืบทอดด้านบนนั้น คลาสแม่คือ `BaseClass` และคลาสลูกคือ `DerivedClass` ซึ่งสืบทอดคุณสมบัติจากคลาสแม่ และเพื่อให้ผู้เรียนได้เห็นถึง

กระบวนการสืบทอดอย่างชัดเจน ผู้เขียนจะยังใช้ตัวอย่างในคลาส Car เช่นเดิม และจะทำการพัฒนาคลาสใหม่ขึ้นมาจากคลาสรถยนต์ คือคลาสรถยนต์ไฟฟ้า หรือคลาส ElectricCar ซึ่งจะได้รับคุณสมบัติรวมทั้งพฤติกรรมมาจากคลาส Car เดิมทั้งหมด

ก่อนที่จะแสดงให้ผู้เรียนเห็นถึงโค้ดโปรแกรมที่ใช้สร้างคลาสรถยนต์ไฟฟ้านั้น อยากให้ผู้เรียนเข้าใจถึงความสัมพันธ์ระหว่างคลาสลูกและคลาสแม่กันก่อน โดยเมื่อเราสร้างอินสแตนซ์จากคลาสลูกนั้น สิ่งแรกที่โปรแกรมภาษาไพธอนจะดำเนินการ คือการกำหนดค่าทั้งหมดให้กับแอตทริบิวต์ในคลาสแม่ด้วยการส่งค่าผ่านเมธอด `__init__()` ของคลาสลูก

```
1. class Car:
2.     ..
3.     ..
4.
5. class ElectricCar(Car):
6.     """คลาสรถยนต์ไฟฟ้าสืบทอดคุณสมบัติจากคลาสรถยนต์"""
7.     def __init__(self, make, model, year):
8.         """กำหนดค่าเริ่มต้นจากพารามิเตอร์ให้กับคลาสแม่"""
9.         super().__init__(make, model, year)
10.
11. my_ecar = ElectricCar('Tesla', 'model s', 2020)
12. print(my_ecar.get_descriptive_name())
```

[output]
2020 Tesla Model S

จากตัวอย่างด้านบนเมื่อเราสร้างคลาสลูก คลาสแม่จะต้องปรากฏอยู่ในไฟล์โค้ดโปรแกรมก่อนคลาสลูกเสมอ และเมื่อเราสร้างคลาสลูกนั้นชื่อของคลาสแม่จะต้องปรากฏอยู่ในเครื่องหมายวงเล็บหลังชื่อของคลาสลูก ดังแสดงในบรรทัดที่ 5

จากบรรทัดที่ 9 ฟังก์ชัน `super()` เป็นฟังก์ชันพิเศษที่ช่วยให้โปรแกรมภาษาไพธอนเชื่อมต่อระหว่างคลาสแม่และคลาสลูก และทำการส่งค่าพารามิเตอร์ `make`, `model` และ `year` ไปกำหนดค่าให้แก่แอตทริบิวต์ของคลาสแม่ผ่านเมธอด `__init__()`

และด้วยการสืบทอดเช่นนี้ จากโค้ดโปรแกรมผู้เรียนจะเห็นว่า แม้ว่าคลาส ElectricCar จะไม่ได้มีการสร้างแอตทริบิวต์ภายในคลาสเลย แต่ก็สามารถเรียกใช้งานแอตทริบิวต์ทั้งหมดได้จากคลาส Car

12.3.3. การกำหนดแอตทริบิวต์และเมธอดให้คลาสลูก

เมื่อทำการสร้างคลาสลูกที่สืบทอดมาจากคลาสแม่แล้ว เราสามารถเพิ่มแอตทริบิวต์และเมธอดใหม่ ๆ ที่จำเป็นต่อการทำงานเข้าไปในคลาสใหม่ที่เราร่างขึ้นมา ซึ่งเป็นการแยกความแตกต่างระหว่างคลาสลูกและคลาสแม่

จากตัวอย่างคลาสรถยนต์ไฟฟ้าที่สืบทอดมาจากคลาสรถยนต์ที่ผ่านมา เราจะทำการเพิ่มแอตทริบิวต์บางอย่างเข้าไปซึ่งจะทำให้คลาสลูกที่สร้างขึ้นมีความแตกต่างจากคลาสแม่ นั่นคือแอตทริบิวต์ขนาดบรรจุแบตเตอรี่ (Battery Size) และสร้างเมธอดใหม่ที่ใช้สำหรับรายงานขนาดบรรจุแบตเตอรี่ของคลาส ดังแสดงในตัวอย่างด้านล่าง

จากตัวอย่างด้านล่างเราเพิ่มแอตทริบิวต์ใหม่คือ `self.battery_size` ให้กับคลาส `ElectricCar` และกำหนดค่าเริ่มต้นให้มีค่าเท่ากับ 100 ซึ่งแอตทริบิวต์นี้จะเชื่อมโยงกับอินสแตนซ์ทั้งหมดที่สร้างจากคลาส `ElectricCar` เท่านั้น แต่จะไม่เชื่อมโยงกับอินสแตนซ์ใด ๆ ที่เกิดจากคลาส `Car` นอกจากนี้เรายังสร้างเมธอด สำหรับพิมพ์ข้อมูลเกี่ยวกับแบตเตอรี่ ซึ่งเมื่อเราเรียกเมธอดผ่านอินสแตนซ์เราจะได้รับคำอธิบายที่ชัดเจนสำหรับรถยนต์ไฟฟ้านั้น

```
1. class ElectricCar(Car):
2.     """คลาสรถยนต์ไฟฟ้าสืบทอดคุณสมบัติจากคลาสรถยนต์"""
3.     def __init__(self, make, model, year):
4.         """กำหนดค่าเริ่มต้นจากพารามิเตอร์ให้กับคลาสแม่"""
5.         super().__init__(make, model, year)
6.         self.battery_size = 100
7.
8.     def describe_battery(self):
9.         """พิมพ์ขนาดบรรจุแบตเตอรี่ออกทางหน้าจอ"""
10.        print("This car has {}".format(self.battery_size))
11.
12.
13. my_ecar = ElectricCar('Tesla', 'model s', 2020)
14. my_ecar.describe_battery()
```

[output]
This car has 100.

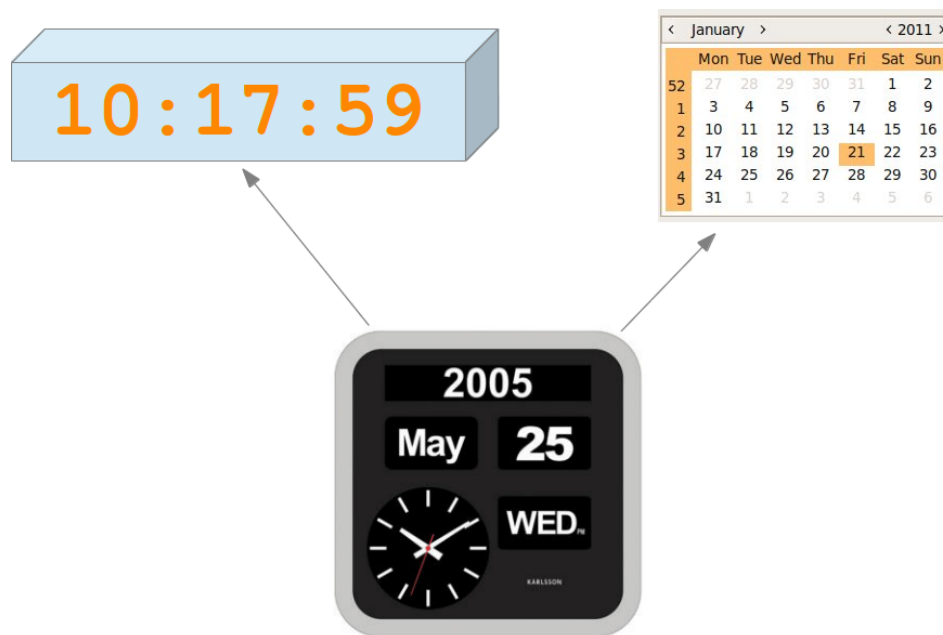
ผู้เรียนจะเห็นว่าเราสามารถเพิ่มเติมแอตทริบิวต์และเมธอดให้กับคลาสลูกซึ่งจะทำให้คลาสลูกมีความแตกต่างจากคลาสแม่จำนวนมากเท่าไรก็ได้โดยไม่มียกเว้น แต่ก็ยังคงลักษณะที่สืบทอดมาจากคลาสแม่เดิมเอาไว้ทุกประการด้วยเช่นกัน และการเพิ่มเติมแอตทริบิวต์และเมธอดใด ๆ ในคลาสลูกนั้นจะไม่ส่งผลย้อนกลับไปยังคลาสแม่แต่อย่างใด

12.3.4. การสืบทอดจากหลายคลาส

ในตัวอย่างก่อนหน้านี้เป็นการสืบทอดคลาสจากเพียงแค่คลาสเดียว อย่างไรก็ตามในภาษาโปรแกรมไพธอนนั้นสนับสนุนการสืบทอดจากหลายคลาส (Multiple inheritance) ได้พร้อมกัน ซึ่งเป็นการสืบทอดแอตทริบิวต์และเมธอดจากคลาสแม่มากกว่าหนึ่งคลาส

เพื่อให้ผู้เรียนได้มองเห็นภาพที่ชัดเจนว่ามีกรณีใดบ้างที่เราสามารถสร้างคลาสใหม่จากคลาสเดิมด้วยการสืบทอดจากหลายคลาส ผู้เขียนจึงขอยกตัวอย่างการจำลองแบบวัตถุนาฬิกาดิจิทัลที่

แสดงปฏิทินบนหน้าปัดได้ (Digital Calendar Clock) ซึ่งการจำแบบวัตถุที่เราต้องการนี้สามารถจำลองได้โดยการนำเอาคลาสนาฬิกาดิจิทัล และคลาสปฏิทินดิจิทัลที่ได้ถูกพัฒนาเอาไว้เรียบร้อยแล้ว มาสร้างเป็นคลาสใหม่ นั่นคือคลาส DigitalCalendarClock ซึ่งสืบทอดแอตทริบิวต์และเมธอดจากทั้งคลาสนาฬิกาดิจิทัล และคลาสปฏิทินดิจิทัล โดยแนวคิดของการจำลองแบบวัตถุที่มีการสืบทอดจากหลายคลาสนี้สามารถแสดงได้ดังภาพที่ 12.1



ภาพที่ 12.1 แนวคิดในการจำลองแบบวัตถุด้วยการสืบทอดจากหลายคลาส

ที่มา: เว็บไซต์ https://www.python-course.eu/python3_multiple_inheritance.php

สามารถแสดงไวยากรณ์ของภาษาไพธอนสำหรับการสืบทอดจากหลายคลาสดังนี้

1. Class Base1:
2. Body of the **class**
- 3.
4. Class Base2:
5. Body of the **class**
- 6.
7. Class Derived(Base1, Base2):
8. Body of the **class**

จากไวยากรณ์ของภาษาไพธอนด้านบน เราจะอธิบายด้วยตัวอย่างอย่างง่ายเพื่อให้ผู้เรียนเข้าใจกลไกการสืบทอดจากหลายคลาส โดยการทดลองสร้างคลาสใหม่คือคลาส Resident ที่สืบทอดทั้งแอตทริบิวเตอร์และเมธอดมาจากคลาส Person และคลาส Student ดังตัวอย่างต่อไปนี้

```

1. class Person:
2.     #defining constructor
3.     def __init__(self, personName, personAge):
4.         self.name = personName
5.         self.age = personAge
6.
7.     #defining class methods
8.     def showName(self):
9.         print(self.name)
10.
11.    def showAge(self):
12.        print(self.age)
13.
14.    #end of class definition
15.
16. class Student:
17.    def __init__(self, studentId):
18.        self.studentId = studentId
19.
20.    def getId(self):
21.        return self.studentId
22.
23.
24. class Resident(Person, Student):
25.    def __init__(self, name, age, id):
26.        Person.__init__(self, name, age)
27.        Student.__init__(self, id)
28.
29.
30. # สร้างอินสแตนซ์จากคลาสลูก
31. resident1 = Resident('John', 30, '102')
32. resident1.showName()
33. print(resident1.getId())

```

[output]

```

John
102

```

12.3.5. การลบล้างหรือแก้ไขเมธอดที่ได้รับสืบทอดจากคลาสแม่

ในการสร้างคลาสลูกด้วยการสืบทอดจากคลาสแม่นั้น หากในคลาสแม่มีเมธอดใด ๆ ที่มีการทำงานไม่ตรงกับความต้องการของเรา หรือหากเราต้องการเปลี่ยนแปลงการทำงานของเมธอดใด ๆ ที่คลาสลูกได้รับการสืบทอดมาจากคลาสแม่ให้ดียิ่งขึ้น เราสามารถทำได้โดยการกำหนดเมธอดในคลาสลูกใหม่ด้วยชื่อเดียวกับเมธอดที่มีอยู่เดิมในคลาสแม่ ด้วยวิธีการนี้ในภาษาโปรแกรม

ไพธอนจะไม่สนใจเมธอดจากคลาสแม่ แต่จะให้ความสนใจเฉพาะเมธอดที่เราสร้างขึ้นใหม่ในคลาสลูกแทน ซึ่งเราจะเรียกว่าการดำเนินการนี้ว่า “Overriding Method”

สมมติว่าคลาสรถยนต์มีเมธอดชื่อว่า `fill_gas_tank()` ใช้สำหรับการเติมน้ำมันให้กับรถยนต์ แต่เมธอดนี้ไม่มีความจำเป็นต้องใช้งานในรถยนต์ไฟฟ้าทั้งหมด ดังนั้นเราอาจต้องทำการลบล้างเมธอดนี้ออกไปให้ไม่มีการทำงานใด ๆ ในคลาสรถยนต์ไฟฟ้า ซึ่งสามารถทำได้ดังตัวอย่างด้านล่าง

```
1. class ElectricCar(Car):
2.     ..
3.     ..
4.     def fill_gas_tank(self):
5.         """ลบล้างเมธอดจากคลาสแม่
6.         เนื่องจากในรถยนต์ไฟฟ้าไม่มีถังสำหรับเติมน้ำมัน"""
7.         print("This car doesn't need a gas tank!")
```

ตอนนี้หากมีคณพยายามเรียก `fill_gas_tank()` จากอินสแตนซ์รถยนต์ไฟฟ้า โปรแกรมภาษาไพธอนจะเพิกเฉยต่อเมธอด `fill_gas_tank()` ในคลาสรถยนต์ซึ่งเป็นคลาสแม่ และเรียกใช้เมธอดที่อยู่ในคลาสรถยนต์ไฟฟ้าแทน เมื่อเราใช้การสืบทอดเราสามารถทำให้คลาสลูกของเราเองไว้ซึ่งสิ่งที่เราต้องการ และสามารถลบล้างสิ่งที่เราไม่ต้องการจากคลาสแม่ได้

12.4. การนำเข้าคลาส

เมื่อเราทำการเพิ่มฟังก์ชันการทำงานให้กับคลาสมากขึ้น โค้ดโปรแกรมที่เราเขียนอาจมีความยาวเพิ่มมากขึ้น และมีความซับซ้อนเพิ่มมากขึ้นตามไปด้วยด้วย ถึงแม้ว่าเราจะใช้การสืบทอดคลาสอย่างเหมาะสมก็ตาม เพื่อให้เป็นไปตามปรัชญาของโปรแกรมภาษาไพธอนที่ได้วางเอาไว้ นั่นคือความเรียบง่ายและสามารถเข้าใจได้ง่าย ดังนั้นเราจะต้องจัดการไฟล์โค้ดโปรแกรมของเราให้มีความกระชับและมีความซับซ้อนน้อยมากที่สุด ดังนั้นโปรแกรมภาษาไพธอนได้อนุญาตให้เราสามารถจัดเก็บคลาสต่าง ๆ เอาไว้ในโมดูล (Module) แล้วนำเข้าคลาสที่เราต้องการใช้งานเข้าไปทำงานในโปรแกรมหลักของเราได้

สำหรับหัวข้อนี้เป็นการแนะนำให้ผู้เรียนได้ทราบถึงเทคนิคการพัฒนาโปรแกรมที่เกี่ยวข้องกับการนำเข้าคลาสจากโมดูล โดยเริ่มจากการพัฒนาคลาสต่าง ๆ และจัดเก็บไว้ในโมดูล และวิธีการนำเข้าคลาสจากโมดูลเข้ามาใช้งานในไฟล์โปรแกรมที่เรา กำลังทำงานอยู่ ด้วยรูปแบบต่าง ๆ ที่ทางโปรแกรมภาษาไพธอนได้จัดเตรียมเอาไว้ให้

12.4.1. การนำเข้าเพียงคลาสเดียว

พิจารณาจากคลาส Car ในตัวอย่างที่ผ่านมา เราจะทำการบันทึกโค้ดโปรแกรมของคลาส Car เอาไว้ในไฟล์ชื่อว่า car.py และภายในไฟล์ car.py นี้จะมีเฉพาะโค้ดโปรแกรมของคลาส Car เพียงเท่านั้นไม่มีโค้ดโปรแกรมอื่นที่นอกเหนือจากคลาส Car แต่อย่างใด ซึ่งเราจะเรียกไฟล์ car.py นี้ว่าโมดูล โดยไฟล์ car.py สามารถแสดงดังตัวอย่างด้านล่าง

car.py

```

1. class Car():
2.     """คลาสสำหรับจำลองรถยนต์"""
3.     def __init__(self, make, model, year):
4.         """กำหนดค่าเริ่มต้นให้กับแอตทริบิวต์ของคลาสรถยนต์"""
5.         self.make = make
6.         self.model = model
7.         self.year = year
8.         self.odometer = 0
9.
10.    def get_descriptive_name(self):
11.        """ส่งคืนคำรายละเอียดต่าง ๆ ของรถยนต์"""
12.        long_name = str(self.year)+" "+self.make+" "+self.model
13.        return long_name.title()
14.
15.    def read_odometer(self):
16.        """แสดงรายละเอียดระยะสะสมของรถยนต์"""
17.        print("This car has " + str(self.odometer) + " miles")
18.
19.    def update_odometer(self, mileage):
20.        """กำหนดค่าระยะทางสะสมใหม่จากพารามิเตอร์"""
21.        if mileage > self.odometer:
22.            self.odometer = mileage
23.        else:
24.            print("คุณไม่สามารถลดค่าระยะสะสมได้")

```

จากนั้นทำการสร้างโค้ดโปรแกรมใหม่แยกออกมาจากไฟล์ car.py ตั้งชื่อว่า my_car.py โดยไฟล์โปรแกรมนี้จะทำการนำเข้าคลาส Car จากโมดูลและสร้างอินสแตนซ์จากคลาส Car

my_car.py

```

1. from car import Car
2.
3. my_new_car = Car('Toyota', 'Altis', 2016)
4.
5. print(my_new_car.get_descriptive_name())
6. my_new_car.read_odometer()

```

จากตัวอย่างด้านบนในไฟล์ my_car.py คำสั่งนำเข้าที่บรรทัด 1 เป็นการบอกโปรแกรมภาษาไพธอนให้ทำการเปิดโมดูล car และนำเข้าคลาส Car เข้ามาทำงานในไฟล์ my_car.py ซึ่งตอนนี้เราสามารถ useClass Car ได้โดยมีผลลัพธ์ของการทำงานเหมือนกันกับที่เราเห็นจากตัวอย่างของบทเรียนก่อนหน้าที่ผ่านมา

12.4.2. การจัดเก็บหลายคลาสในโมดูล

จากเนื้อหาในหัวข้อที่ 12.4.1 เป็นการจัดเก็บคลาสเพียงคลาสเดียวไว้ในโมดูล แต่อย่างไรก็ตามภายในโมดูลหนึ่งโมดูลนั้น เราจะสามารถจัดเก็บคลาสไว้ในโมดูลนั้น ๆ ได้มากเท่าใดก็ได้ตามที่เรต้องการ โดยตัวอย่างด้านล่างนี้เราจะทำการเพิ่มคลาสแบตเตอรี่ และรถยนต์ไฟฟ้าเข้าไปในไฟล์ car.py

car.py

```

1. class Car():
2.     """คลาสสำหรับจำลองรถยนต์"""
3.     ..
4.     ..
5.
6. class Battery():
7.     def __init__(self, battery_size=60):
8.         self.battery_size = battery_size
9.
10.    def describe_battery(self):
11.        print("This car has a "+str(self.battery_size)+"-kWh")
12.
13.    def get_range(self):
14.        range = 0
15.        if self.battery_size <= 70:
16.            range = 240
17.        elif self.battery_size > 70:
18.            range = 270
19.
20.        message = "This car can go approximately " + str(range)
21.        message += " miles on a full charge."
22.        print(message)
23.
24.
25. class ElectricCar(Car):
26.     """คลาสรถยนต์ไฟฟ้าสืบทอดคุณสมบัติจากคลาสรถยนต์"""
27.     def __init__(self, make, model, year):
28.         """กำหนดค่าเริ่มต้นจากพารามิเตอร์ให้กับคลาสแม่"""
29.         super().__init__(make, model, year)
30.         self.battery = Battery()

```

จากนั้นเราสามารถสร้างไฟล์ใหม่ชื่อ `my_electric_car.py` นำเข้าคลาส `ElectricCar` และสร้างอินสแตนซ์ของรถยนต์ไฟฟ้าคือ `my_ecar` และเรียกใช้เมธอดภายใน `my_ecar` ดังตัวอย่างด้านล่างนี้

`my_electric_car.py`

```
1. from car import ElectricCar
2.
3. my_ecar = ElectricCar('Tesla', 'model s', 2020)
4.
5. print(my_ecar.get_descriptive_name())
6.
7. my_ecar.battery.describe_battery()
8.
9. my_ecar.battery.get_range()
```

[output]

```
2020 Tesla Model S
This car has a 60-kWh
This car can go approximately 240 miles on a full charge.
```

12.4.3. การนำเข้าหลายคลาสจากโมดูล

เราสามารถนำเข้าคลาสได้มากเท่าที่ต้องการใช้งานในไฟล์โปรแกรม หากเราต้องการสร้างรถยนต์ธรรมดาและรถยนต์ไฟฟ้าในไฟล์เดียวกันเราต้องนำเข้าทั้งคลาส `Car` และ `ElectricCar` ดังตัวอย่างต่อไปนี้

`my_cars.py`

```
1. from car import Car, ElectricCar
2.
3. my_beetle = Car('volkswagen', 'beetle', 2016)
4. print(my_beetle.get_descriptive_name())
5.
6. my_tesla = ElectricCar('tesla', 'roadster', 2016)
7. print(my_tesla.get_descriptive_name())
```

[output]

```
2016 Volkswagen Beetle
2016 Tesla Roadster
```

เรานำเข้าหลายคลาสจากโมดูลเดียวกัน ด้วยการแยกแต่ละคลาสโดยใช้เครื่องหมายจุลภาค (",") ดังแสดงในบรรทัดที่ 1 เมื่อเรานำเข้าคลาสที่จำเป็นแล้ว เราสามารถสร้างอินสแตนซ์ของแต่ละคลาสได้มากเท่าที่เราต้องการ

12.4.4. การนำเข้าทั้งโมดูล

นอกการนำเข้าคลาสจากโมดูลด้วยการระบุคลาสที่เราต้องการนำมาใช้งานอย่างเจาะจง แล้วดังตัวอย่างที่ผ่านมา เรายังสามารถนำเข้าโมดูลทั้งหมด จากนั้นจึงค่อยทำการเข้าถึงคลาสที่เราต้องการโดยใช้สัญลักษณ์จุด (“.”) แนวทางนี้เรียบง่ายและให้ผลลัพธ์ในโค้ดที่อ่านง่าย เนื่องจากทุกที่ที่มีการใช้งานคลาสเพื่อสร้างอินสแตนซ์ของคลาสจะมีชื่อโมดูลปรากฏร่วมกันเสมอ และจะทำให้ไม่มีความขัดแย้งในการตั้งชื่อกับชื่อใด ๆ ที่ใช้ในไฟล์โค้ดปัจจุบัน

ลักษณะของการนำเข้าคลาสทั้งหมดจากโมดูล Car เพื่อนำมาสร้างอินสแตนซ์รถยนต์ธรรมดาและรถยนต์ไฟฟ้า สามารถทำได้ดังตัวอย่างต่อไปนี้

```
1. import car
2.
3. my_beetle = car.Car('volkswagen', 'beetle', 2016)
4.
5. my_tesla = car.ElectricCar('tesla', 'roadster', 2016)
```

12.4.5. การนำเข้าคลาสทั้งหมดจากโมดูล

เราสามารถนำเข้าทุกคลาสภายในโมดูลโดยใช้ไวยากรณ์ต่อไปนี้

```
1. from module_name import *
```

ถึงแม้ว่าการนำเข้าคลาสทั้งหมดจากโมดูลจะเป็นวิธีที่ดูเหมือนจะทำได้ง่ายสะดวกต่อการพัฒนาโปรแกรม แต่ผู้เขียนไม่แนะนำให้ใช้วิธีนี้ ด้วยเหตุผลสองประการดังต่อไปนี้ 1) จะเป็นการดีกว่าหากในการนำเข้าคลาสจากโมดูลเป็นไปอย่างชัดเจน ว่าโปรแกรมหลักที่เรากำลังทำงานอยู่นั้นต้องการใช้งานคลาสใดจากโมดูล แต่ด้วยวิธีนี้ทำให้ไม่มีความชัดเจนว่าเราใช้งานคลาสใดจากโมดูลที่เรานำเข้า และเหตุผลประการที่ 2) ด้วยวิธีนี้ยังมีโอกาสนำไปสู่ความสับสนกับการตั้งชื่อคลาสใหม่ภายในไฟล์ปัจจุบันที่เรากำลังทำงานอยู่อีกด้วย หากเราสร้างคลาสใหม่ที่มีชื่อเดียวกับคลาสเดิมที่มีอยู่แล้วในโมดูลที่เรานำเข้าโดยไม่ได้ตั้งใจ สิ่งนี้สามารถสร้างข้อผิดพลาดที่ยากต่อการวินิจฉัยเพื่อแก้ไขความผิดพลาดที่เกิดขึ้นได้

ดังนั้นผู้เขียนจึงมีข้อเสนอแนะว่าสำหรับการนำเข้าคลาสจากโมดูลนั้นควรนำเข้าแบบทั้งโมดูล และใช้ไวยากรณ์ `module_name.class_name` ดังหัวข้อที่ 12.4.4 ซึ่งเราจะไม่เห็นคลาสทั้งหมดที่ถูกนำเข้ามาใช้งานที่ด้านบนของไฟล์โค้ดโปรแกรม แต่จะเห็นได้อย่างชัดเจนว่าโมดูลถูกใช้ในโปรแกรม นอกจากนี้ยังสามารถหลีกเลี่ยงความขัดแย้งในการตั้งชื่อที่อาจเกิดขึ้นเมื่อทำการนำเข้าทุกคลาสจากโมดูลได้อีกด้วย

12.5. สรุป

เนื้อหาในบทเรียนนี้ผู้เรียนได้เรียนรู้แนวคิดการพัฒนาโปรแกรมเชิงวัตถุเบื้องต้น แนวคิดในการจำลองสิ่งต่าง ๆ ตลอดจนเหตุการณ์ที่เราสนใจให้อยู่ในรูปแบบโปรแกรมคอมพิวเตอร์ด้วยการใช้คลาส ได้เรียนรู้วิธีการสร้างคลาสในโปรแกรมภาษาไพธอน การจัดเก็บข้อมูลภายในคลาสด้วยแอตทริบิวต์ ตลอดจนวิธีการเขียนโปรแกรมที่จะทำให้คลาสที่เราสร้างขึ้นมีพฤติกรรมตามสิ่งที่เราจำลองแบบด้วยการสร้างเมธอดซึ่งเป็นฟังก์ชันภายในคลาส ผู้เรียนได้เรียนรู้การเขียนเมธอด `__init__()` ซึ่งเป็นเมธอดที่ถูกเรียกใช้โดยอัตโนมัติเมื่อเรานำคลาสไปสร้างอินสแตนซ์ ผู้เรียนได้เรียนรู้ถึงวิธีการแก้ไขแอตทริบิวต์ของอินสแตนซ์โดยตรง และแก้ไขผ่านเมธอดต่าง ๆ จากภายในคลาส ได้เรียนรู้ถึงกระบวนการสืบทอดที่เราสามารถสร้างคลาสใหม่จากคลาสเดิมที่มีอยู่ และสามารถเพิ่มเติมทั้งแอตทริบิวต์และเมธอดที่ต้องการให้กับคลาสลูกที่สร้างขึ้นใหม่ได้โดยไม่มีผลกระทบกับการทำงานของคลาสแม่ที่เป็นต้นแบบ และสำหรับในคลาสลูกนั้นก็สามารถแก้ไขหรือลบล้างเมธอดที่ได้รับสืบทอดจากคลาสแม่ได้ หากเมธอดนั้นไม่มีความจำเป็น หรือเราต้องการเปลี่ยนแปลงการทำงานของเมธอดเดิมที่ได้รับสืบทอดมาให้มีการทำงานที่แตกต่างออกไปจากคลาสแม่

ผู้เรียนได้เห็นถึงการจัดเก็บคลาสในโมดูลและการนำเข้าคลาสที่ต้องการจากไฟล์โปรแกรมอื่น ๆ ที่ต้องการใช้งาน สามารถจัดระเบียบโครงสร้างโปรแกรมที่เราพัฒนาให้มีความเรียบง่าย และสะดวกต่อการทำความเข้าใจ ซึ่งจะเป็นประโยชน์อย่างมากหากผู้เรียนทำการพัฒนาโปรแกรมที่มีขนาดใหญ่และมีความซับซ้อนมากยิ่งขึ้นในอนาคต

12.6. คำถามท้ายบท

1. จงอธิบายว่า คลาส (Class) คืออะไรและมีองค์ประกอบอะไรบ้าง
2. จงอธิบายว่า อินสแตนซ์ คืออะไร
3. จงอธิบายว่า เมธอด คืออะไร
4. จงอธิบายว่า แอตทริบิว คืออะไร
5. จงอธิบายถึงความสัมพันธ์ระหว่าง คลาส และ วัตถุ
6. จงอธิบายถึงการแก้ไขค่าแอตทริบิวต์ของอินสแตนซ์ ว่าสามารถทำได้ด้วยวิธีใดบ้าง
7. จงอธิบายถึงวิธีการลบล้างหรือแก้ไขเมธอดที่ได้รับสืบทอดจากคลาสแม่ว่าสามารถทำได้อย่างไร
8. จงอธิบายว่า การสืบทอด คืออะไร และมีความสำคัญอย่างไรในการเขียนโปรแกรมเชิงวัตถุ

12.7. บรรณานุกรม

Matthes, E. (2015). **Python Crash Course A Hands-On, Project-Based Introduction to Programming**. United States of America: No Starch Press.

Inheritance .(2020). Retrieved Nov 9, 2020, from
<http://marcuscode.com/lang/python/inheritance>

Multiple Inheritance .(2020). Retrieved Nov 9, 2020, from จาก https://www.python-course.eu/python3_multiple_inheritance.php